

POSTGRE SQL

```
create table customer_data (  
customer_id integer,  
credit_score integer,  
country varchar(50),  
gender varchar(50),  
age integer,  
tenure integer,  
balance numeric(10,2),  
products_number integer,  
credit_card integer,  
active_member integer,  
estimated_salary numeric(10,2),  
churn integer  
)
```

```
select * from customer_data
```

```
-- REMOVING COLUMN
```

```
ALTER TABLE customer_data  
DROP COLUMN estimated_salary
```

```
-- CATEGORISING DATA
```

```
ALTER TABLE customer_data  
ADD COLUMN products varchar(50)
```

```
UPDATE customer_data  
SET products = (SELECT  
CASE WHEN products_number = 1 THEN 'prod_1'  
      WHEN products_number = 2 THEN 'prod_2'  
      WHEN products_number = 3 THEN 'prod_3'  
      ELSE 'prod 4'  
END AS products)
```

```
UPDATE customer_data  
SET products = REPLACE(products, 'prod 4', 'prod_4')
```

age integer	tenure integer	balance numeric (10,2)	credit_card integer	active_member integer	churn integer	products character varying (50)
58	2	133745.44	1	0	1	prod_4
29	6	0.00	1	0	0	prod_1
37	0	163607.18	0	1	0	prod_1
25	5	157780.84	1	1	0	prod_2


```

ELSE 'churned'
END AS churn_status)

```

```

ALTER TABLE customer_data
DROP churn

```

Data Output Messages Notifications										
	ring (50)	gender character varying (50)	age integer	tenure integer	balance numeric (10,2)	products character varying (50)	credit_card_status character varying (50)	activity_status character varying (50)	churn_status character varying (50)	
1		Male	51	4	40685.92	prod_1	owned	active	not churned	
2		Male	40	2	0.00	prod_2	owned	active	not churned	
3		Male	35	5	136857.00	prod_1	not owned	inactive	not churned	
4		Female	27	9	152328.88	prod_2	not owned	inactive	not churned	

CATEGORISING DATA

```

-- Creating the age_groups column
ALTER TABLE customer_data
ADD age_groups VARCHAR(50)
-- Populating the age_groups column based on the values in the age column
UPDATE customer_data
SET age_groups = (SELECT CASE WHEN age <= 20 THEN '< 20'
                           WHEN age <= 30 THEN '21 - 30'
                           WHEN age <= 40 THEN '31 - 40'
                           WHEN age <= 50 THEN '41 - 50'
                           WHEN age <= 60 THEN '51 - 60'
                           WHEN age <= 70 THEN '61 - 70'
                           ELSE '> 71'
END AS age_groups)
-- Creating a reference table called age_groups
-- from the customerdata table (duplicating table in Power BI)
select age_groups into age_groups from customer_data
-- Removing Duplicates in the age_groups column of the age_groups table
-- Deleting from a CTE is not possible PostgreSQL. However the relational DB
-- has a special system column called ctid that can temporarily identify each physical row.
-- I used it to delete the duplicates:
WITH cte AS (SELECT
ctid,
age_groups,
ROW_NUMBER() OVER (PARTITION BY age_groups ORDER BY ctid) AS rn
FROM age_groups)

DELETE FROM age_groups
WHERE ctid IN (SELECT ctid FROM cte
WHERE rn > 1);
-- Adding the column age_groups_ID in the age_groups table

```

```

ALTER TABLE age_groups
ADD age_groups_ID INTEGER
-- Sorting (re-ordering) the age_groups column of the age_groups table
UPDATE age_groups
SET age_groups_ID = (SELECT CASE WHEN age_groups = '< 20' THEN 1
                                WHEN age_groups = '21 - 30' THEN 2
                                WHEN age_groups = '31 - 40' THEN 3
                                WHEN age_groups = '41 - 50' THEN 4
                                WHEN age_groups = '51 - 60' THEN 5
                                WHEN age_groups = '61 - 70' THEN 6
                                ELSE 7
                                END AS age_groups_ID)

```

```
select * from age_groups
```

Data Output		Messages	Notifications
	age_groups character varying (50) 🔒	age_groups_id integer 🔒	
1	31 - 40	3	
2	41 - 50	4	
3	21 - 30	2	
4	61 - 70	6	
5	51 - 60	5	
6	> 71	7	
7	< 20	1	

```

-- Creating the credit_scores column
select * from customer_data
alter table customer_data
add credit_scores varchar(50)
-- Populating the credit_scores column based on the values in the credit_score column
update customer_data
set credit_scores = (select case when credit_score <= 400 then '<= 400'
                                when credit_score <= 500 then '401 - 500'
                                when credit_score <= 600 then '501 - 600'
                                when credit_score <= 700 then '601 - 700'
                                when credit_score <= 800 then '701 - 800'
                                when credit_score >= 800 then '>= 800'
                                end as credit_scores)
-- Creating a reference table called credit_score_groups

```


Data Output		Messages	Notifications
	credit_scores character varying (50) 🔒	credit_score_id integer 🔒	
1	701 - 800		5
2	601 - 700		4
3	501 - 600		3
4	401 - 500		2
5	>= 800		6
6	<= 400		1

-- Creating the account_balance column

```
ALTER TABLE customer_data
```

```
ADD account_balance VARCHAR(50)
```

-- Populating the account_balance column based on the values in the balance column

```
update customer_data
```

```
set account_balance = (select case when balance = 0 then '0'
```

```
    when balance <= 1000 then '1 - 1k'
```

```
    when balance <= 10000 then '1k - 10k'
```

```
    when balance <= 100000 then '10k - 100k'
```

```
    when balance <= 200000 then '100k - 200k'
```

```
    when balance > 200000 then '> 200k'
```

```
end as account_balance)
```

-- Creating a reference table called Account_balance_groups

-- from the customer_data table (duplicating table in Power BI)

```
select account_balance into account_balance_groups from customer_data
```

```
select * from account_balance_groups
```

Data Output		Messages	Notifications
	account_balance character varying (50) 🔒		
1	0		
2	100k - 200k		
3	0		
4	100k - 200k		
5	0		
6	0		
7	100k - 200k		

```
-- Removing Duplicates in the account_balance column of the account_balance_groups table
-- Deleting from a CTE is not possible PostgreSQL. However the relational DB
-- has a special system column called ctid that can temporarily identify each physical row.
-- I used it to delete the duplicates:
```

```
WITH cte AS (SELECT
ctid,
account_balance,
ROW_NUMBER() OVER (PARTITION BY account_balance ORDER BY ctid) AS rn
FROM account_balance_groups)
```

```
DELETE FROM account_balance_groups
WHERE ctid IN (SELECT ctid FROM cte
WHERE rn > 1);
```

```
-- Adding the column account_balance_ID in the account_balance_groups table
ALTER TABLE account_balance_groups
ADD account_balance_ID INTEGER
-- Sorting (re-ordering) the credit_scores column of the credit_score_groups table
```

```
select * from account_balance_groups
UPDATE account_balance_groups
SET account_balance_ID = (SELECT CASE WHEN account_balance = '0' THEN 1
                                WHEN account_balance = '1k - 10k' THEN 2
                                WHEN account_balance = '10k - 100k' THEN 3
                                WHEN account_balance = '100k - 200k' THEN 4
                                WHEN account_balance = '> 200k' THEN 5
                                END AS account_balance_id)
```

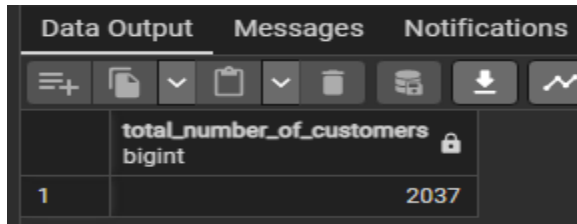
Data Output			Messages	Notifications
	account_balance character varying (50)	account_balance_id integer		
1	0	1		
2	100k - 200k	4		
3	10k - 100k	3		
4	> 200k	5		
5	1k - 10k	2		

REPLICATING DAX MEASURES IN SQL

```
select * from customer_data
SELECT COUNT(customer_id) AS total_number_of_customers FROM customer_data
```

-- total number of customers lost

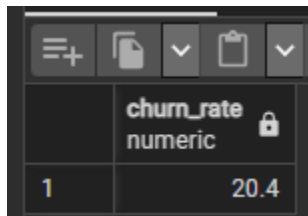
```
SELECT COUNT(customer_id) AS total_number_of_customers FROM customer_data
WHERE churn_status = 'churned'
```



	total_number_of_customers bigint
1	2037

-- churn rate

```
SELECT ROUND(COUNT(customer_id) * 100.00 / (SELECT COUNT(customer_id) FROM customer_data),1) AS
churn_rate
FROM customer_data
WHERE churn_status = 'churned'
```



	churn_rate numeric
1	20.4

POWER BI VISUALS REPLICATED IN SQL

-- Total Number of Customers and percentage by gender

```
select * from customer_data
```

```
select gender, COUNT(customer_id) AS count_of_customers,
```

```
ROUND(COUNT(customer_id) * 100.00 / (select COUNT(customer_id) from customer_data),2)
```

```
AS pct
```

```
from customer_data
```

```
group by gender
```

-- Total Number of Customers and percentage by activity_status

```
select activity_status, COUNT(customer_id) AS count_of_customers,
```

```
ROUND(COUNT(customer_id) * 100.00 / (select COUNT(customer_id) from customer_data),2)
```

```
AS pct
```

```
from customer_data
```

```
group by activity_status
```

```
-- Total Number of Customers and percentage by credit_card_status

select credit_card_status, COUNT(customer_id) AS count_of_customers,

ROUND(COUNT(customer_id) * 100.00 / (select COUNT(customer_id) from customer_data),2)

AS pct

from customer_data

group by credit_card_status
```

Data Output Messages Notifications			
	gender character varying (50) 🔒	count_of_customers bigint 🔒	pct numeric 🔒
1	Female	4543	45.43
2	Male	5457	54.57

Data Output Messages Notifications			
	activity_status character varying (50) 🔒	count_of_customers bigint 🔒	pct numeric 🔒
1	inactive	4849	48.49
2	active	5151	51.51

Data Output Messages Notifications			
	credit_card_status character varying (50) 🔒	count_of_customers bigint 🔒	pct numeric 🔒
1	not owned	2945	29.45
2	owned	7055	70.55

```
-- Total number of customers and churn rate (avg churn per customer)

-- broken down by age_groups and sorting (age_groups_id)

select ag.age_groups, ag.age_groups_id,

COUNT(c.customer_id) AS total_number_of_customers,

SUM(CASE WHEN c.churn_status = 'churned' THEN 1 ELSE 0 END) * 100.00 /

COUNT(c.customer_id) AS AVG_churn_per_customer

from customer_data c

inner join age_groups ag on ag.age_groups = c.age_groups

group by ag.age_groups, ag.age_groups_id

order by ag.age_groups_id
```

Data Output

Messages

Notifications

≡+

	age_groups character varying (50)	age_groups_id integer	total_number_of_customers bigint	avg_churn_per_customer numeric
1	< 20	1	89	5.6179775280898876
2	21 - 30	2	1879	7.6104310803618946
3	31 - 40	3	4451	12.0871714221523253
4	41 - 50	4	2320	33.9655172413793103
5	51 - 60	5	797	56.2107904642409034
6	61 - 70	6	331	31.4199395770392749
7	> 71	7	133	8.2706766917293233

-- Total number of customers and AVG churn per customer

-- broken down by credit scores and credit scores ID

```
select csg.credit_scores, csg.credit_score_id,
COUNT(c.customer_id) AS total_number_of_customers,
ROUND(SUM(CASE WHEN c.churn_status = 'churned' THEN 1 ELSE 0 END) * 100.00 /
COUNT(c.customer_id),2) AS AVG_churn_per_customer
from customer_data c
inner join credit_score_groups csg on csg.credit_scores = c.credit_scores
group by csg.credit_scores, csg.credit_score_id
order by csg.credit_score_id
```

Data Output

Messages

Notifications

	credit_scores character varying (50)	credit_score_id integer	total_number_of_customers bigint	avg_churn_per_customer numeric
1	<= 400	1	19	100.00
2	401 - 500	2	624	21.31
3	501 - 600	3	2423	21.17
4	601 - 700	4	3818	19.72
5	701 - 800	5	2471	19.91
6	>= 800	6	645	19.69

-- Total number of customers and churn rate

-- broken down by account balance and account balance ID

```
select abg.account_balance, abg.account_balance_id,
COUNT(c.customer_id) AS count_of_customers,
```

```

SUM(CASE WHEN churn_status = 'churned' THEN 1 ELSE 0 END) * 100.00 /
COUNT(c.customer_id) AS churn_rate
from customer_data c
inner join account_balance_groups abg on c.account_balance = abg.account_balance
group by abg.account_balance, abg.account_balance_id
order by abg.account_balance_id

```

Data Output Messages Notifications				
	account_balance character varying (50)	account_balance_id integer	count_of_customers bigint	churn_rate numeric
1	0	1	3617	13.8236107271219242
2	1k - 10k	2	1	100.0000000000000000
3	10k - 100k	3	1583	20.5306380290587492
4	100k - 200k	4	4765	25.0157397691500525
5	> 200k	5	34	55.8823529411764706

-- ADD THE TARGET CHURN RATE COLUMN

```
ALTER TABLE customer_data
```

```
ADD target_churn_rate DECIMAL(10,2)
```

-- SET A TARGET RATE OF 0.15 (15%)

```
UPDATE customer_data
```

```
SET target_churn_rate = 0.15
```

-- AVG CHURN RATE PER CUSTOMER AND TARGET_CHURN_RATE IN PCT

```
select target_churn_rate,
```

```
SUM(CASE WHEN churn_status = 'churned' THEN 1 ELSE 0 END) * 100.00 /
```

```
(SELECT COUNT(customer_id) from customer_data) AS AVG_churn_rate_per_customer ,
```

```
target_churn_rate * 100.00 AS target_churn_rate_pct
```

```
FROM customer_data
```

```
group by target_churn_rate
```

Data Output Messages Notifications			
	target_churn_rate numeric (10,2)	avg_churn_rate_per_customer numeric	target_churn_rate_pct numeric
1	0.15	20.3700000000000000	15.0000

